

Implementasi Deep Reinforcement Learning dengan Algoritma PPO pada Lingkungan Grid-Based untuk Permainan Nokia Snake Game

1. Project Overview

Implementasi Deep Reinforcement Learning untuk navigasi lingkungan grid dinamis melalui modifikasi permainan Snake klasik pada nokia. Tantangan utama terdapat dalam navigasi agen untuk dapat melakukan penghindaran Greedy Trap kondisi di mana agen terjebak oleh tubuhnya sendiri dan pelacakan target yang bergerak.

2. Paper Reference

Judul Paper: [Deep Q-Snake: An Intelligent Agent Mastering the Snake Game with Deep Reinforcement Learning](#)

Penulis: Debjyoti Ray, Arindam Ghosh, Muneendra Ojha, Krishna Pratap Singh (IIIT Allahabad)

Tahun / Konferensi: 2024 / IEEE Region 10 Conference (TENCON)

Arsitektur dan State Space pada Paper Referensi:

Didefinisikan state space menggunakan himpunan 12-bit biner yang terdiri dari:

Status Makanan (4 bit): Makanan di Atas, Bawah, Kiri, Kanan.

Arah Agen (4 bit): Ular bergerak ke Atas, Bawah, Kiri, Kanan.

Status Dinding/Rintangan (4 bit): Terdapat dinding di Atas, Bawah, Kiri, Kanan.

Untuk memproses 12 fitur biner tersebut, Arsitektur neural network yang digunakan dalam paper dibangun menggunakan Keras dengan Model Sequential (Multi-Layer Perceptron / Dense Layers):

Input Layer: Menerima 12 digit state biner.

Hidden Layers: Terdiri dari dua lapis Dense/Fully Connected Layer menggunakan fungsi aktivasi ReLU untuk memperkenalkan non-linearitas.

Output Layer: Dense layer dengan 4 neuron yang merepresentasikan nilai Q-Value untuk masing-masing aksi diskrit (Atas, Bawah, Kiri, Kanan).

3. Proposed Architecture

Peta: $M \times N$

State Space $M \times N \times 4$:

Channel 1: Peta biner rintangan/dinding batas matriks.

Channel 2: Representasi tubuh agen menggunakan fungsi degradasi matematis ($1.0 - (\text{index}/\text{panjang_ular})$). Kepala bernilai 1.0, menurun secara linear mendekati 0.0 di ujung ekor.

Channel 3 (Target Statis): Matriks biner $M \times N$ untuk koordinat makanan statis.

Kanal 4 (Momentum Dinamis): Jejak temporal dari makanan dinamis (posisi saat ini bernilai 1.0, posisi t-1 bernilai 0.5) untuk memberikan proyeksi vektor gerak ke dalam arena spasial.

Architecture:

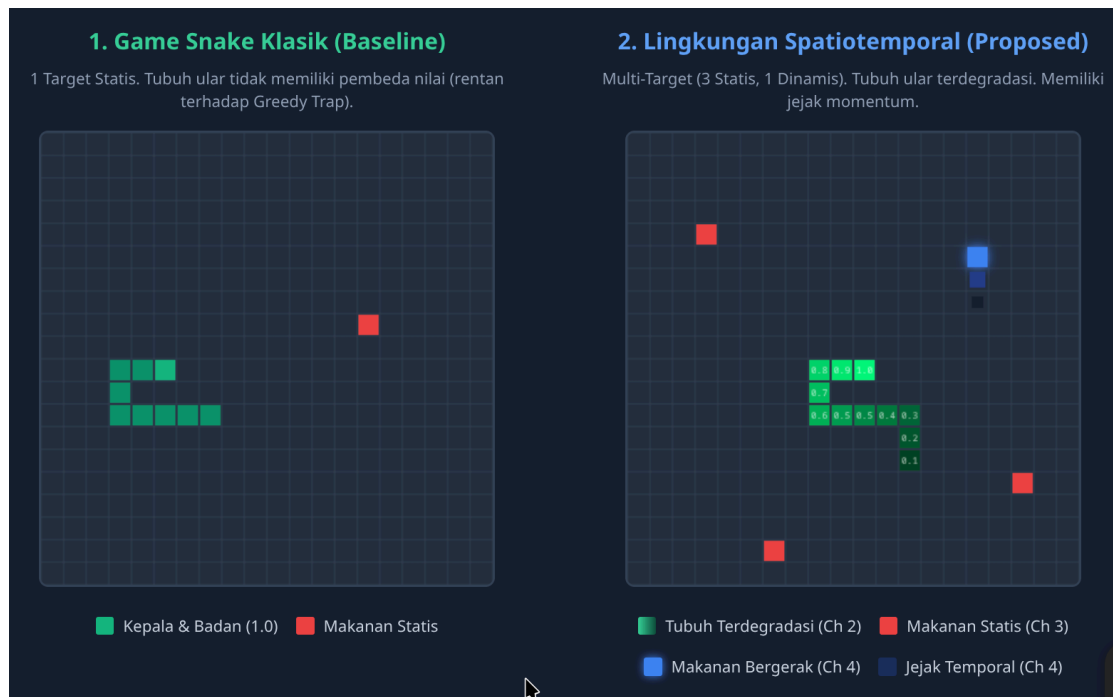
- Engine A - Spatial Feature Extractor (CNN): Menggantikan arsitektur Dense Layer primitif dari paper referensi.
 - Conv2D_1: 32 filter, kernel 3x3, stride 1, ReLU.
 - Conv2D_2: 64 filter, kernel 3x3, stride 1, ReLU.
 - Flatten Layer: Mengubah matriks spasial menjadi representasi fitur 1D berdimensi 256.
- Engine B - Temporal Attention Block (Transformer Encoder): Fitur spasial diteruskan ke mekanisme Self-Attention berbobot ringan untuk memecahkan Temporal Blindness.
 - Architecture: 1 Layer Transformer Encoder dengan 4 Attention Heads. Menghitung relevance score mengizinkan agen untuk "mengabaikan" makanan statis yang jauh dan memfokuskan komputasi pada trajektori makanan yang sedang bergerak berdasarkan data dari Channel 4 pada data SpasioTemporal 4 Channel.
- Engine C - Actor-Critic Heads: Output dari Attention Block diumpangkan ke kepala PPO:
 - Policy Head (Actor): Memprediksi probabilitas distribusi dari 4 aksi Discrete dengan mekanisme Clipping.
 - Value Head (Critic): Mengestimasi State-Value Function $V(s)$ untuk mengkalkulasi Advantage.

4. Platform Implementasi

Environment Physics & Logic: Lingkungan game dibuat secara custom dengan Pygame untuk manipulasi susunan matriks grid dasar dan perhitungan pergerakan (kinematika).

RL Standardization Wrapper: Gymnasium (OpenAI Gym). Logika Pygame dibungkus ke dalam gym.Env dengan spaces.Box berukuran (M, N, 4) sebagai Observation Space. Mengimplementasikan teknik paralelisasi SubprocVecEnv (4 parallel environments) untuk memastikan komputasi CPU dimanfaatkan penuh secara sinkron.

Detail permainan: Tambahan modifikasi untuk permainan dimana pada game klasik snake game hanya terdapat 1 makanan yang dapat muncul pada peta, untuk lingkungan yang dimodifikasi akan setidaknya terdapat 4 makanan yang muncul pada peta ditambah dengan 1 makanan yang dapat bergerak secara acak. Makanan yang bergerak secara acak memiliki 2 state yaitu acak dan menjauh(menjauhi ular).



Detail tambahan aksi makanan dinamis meliputi gerakan acak 2 state:

State 1: variable acak untuk jauh gerak lurus dari makanan

State 2: variable acak untuk arah dari gerak makanan (dalam state ini juga dari 4 arah gerak makanan terdapat tambahan state ke 5 untuk sebuah state dimana arah makanan tidak acak tetapi secara aktif menjauh dari ular)

5. Metodologi

Desain Environment: Memprogram matriks internal Pygame dan memvalidasi kebenaran perhitungan output fungsi degradasi tubuh (decaying tail) secara linear pada tensor channel 2 setiap kali agen melangkah.

Rekayasa Reward (Reward Shaping): Menetapkan struktur hadiah secara hati-hati melalui komputasi aljabar skalar misal: +10.0 untuk memakan target statis, +0.5 untuk mendekati target baik dinamis dan statis, +13 untuk mencegat dan memakan target dinamis, -10.0 terminal penalty menabrak rintangan/tubuh, dan pinalti iterasi sekecil -0.01 per langkah untuk memicu efisiensi optimalitas rute jalan.

Training Iteration: Melatih agen dalam vectorized environment untuk mensintesis jutaan metrik langkah secara mandiri (Headless).

Evaluasi dan Komparasi: Mengekstrak checkpoints model (.zip) secara periodik dan membandingkan model final terhadap arsitektur eksperimen lainnya.

6. Rencana Eksperimen

Eksperimen Lingkungan: Menguji agen pada 4 tingkat kompleksitas permainan, Tingkat 1 1 Makanan statis pada peta, Tingkat 2 Lebih dari 1 makanan statis pada peta, Tingkat 3 1 Makanan yang dapat bergerak, Tingkat 4 Lebih dari 1 makanan statis dan 1 makanan dinamis yang dapat bergerak pada peta, skenario tambahan untuk seluruh makanan bertipe dinamis dengan jumlah lebih dari 1

Eksperimen Arsitektur: Menguji agen dengan menghilangkan block attention pada arsitektur model.

Eksperimen Hyperparameter: Menguji agen yang di train dengan parameter berbeda.

Untuk eksperimen lingkungan akan dilakukan dengan kedua model DQN dengan 12bit state-space dan PPO dengan spatiotemporal $M \times N \times 4$ state-space